

Combined Physical Design + STA Interview Notes

CTS Optimization: Build Clock → Route Clock → Optimize Clock

Scope: practical interview-oriented notes covering latency, skew, clock crosstalk/SI, CTS stages, and the techniques: skew groups, clock balance groups, ICG, CCD, CDC, NDR, and shielding. Commands are representative ICC2-style examples; exact option names and legal combinations can vary by ICC2 release and flow methodology.

1. CTS Mental Model

Clock tree goal: deliver the clock to all sequential sinks with controlled insertion delay (latency), small and intentional skew, acceptable transition/capacitance/fanout, and robust behavior under extraction, variation and SI.

Clock path: source/root → clock network cells → clock routing → sink pin. CTS creates/reshapes the network; routing adds real RC and coupling; clock optimization repairs timing and electrical/SI problems.

Metric	Meaning	CTS concern	Typical symptom
Latency / insertion delay	Time from clock source/reference point to a sink	Keep within target and avoid unnecessary depth/RC	Large clock delay; setup/hold shifts
Skew	Difference in arrival time between related sinks	Balance relevant sinks; do not blindly minimize all skew	Hold/setup imbalance, clock reconvergence issues
Transition	Clock slew at a pin	Prevent slow clock edges	DRV violations, SI sensitivity
Capacitance / fanout	Load driven by clock cell/network	Limit load and distribute buffering	Overloaded clock cells, slow edges
Crosstalk / SI	Neighbor switching changes clock delay/waveform	Control coupling, routing and aggressor effects	Post-route skew changes, jitter-like uncertainty, noise

2. CTS Stages — Build Clock → Route Clock → Optimize Clock

BUILD CLOCK

Create the clock topology. Identify clock roots and sinks, cluster sinks, insert buffers/inverters, manage fanout/cap/transition, and establish balance targets/groups. At this point routing is usually abstract or estimated.

Interview point: If topology is poor, routing and optimization inherit the problem.

ROUTE CLOCK

Implement the clock network on physical metal. Choose routing layers/rules, NDRs and shielding where required, then analyze extracted RC and coupling.

Interview point: Post-route RC/coupling can change latency and skew compared with build estimates.

OPTIMIZE CLOCK

Repair timing/electrical/SI issues by resizing/inserting/removing clock cells, changing topology/balance, and applying useful-skew/CCD-style optimization where enabled.

Interview point: The objective is not simply 'minimum skew'; it is robust timing closure with acceptable latency, slew, cap and SI across scenarios.

3. BUILD CLOCK — What Happens and What to Watch

Root and sink definition

Clock roots are the source points; sinks are sequential clock pins and relevant endpoints. Incorrect sink classification or exclusions can create misleading balance.

Clustering

Nearby sinks are grouped so the tool can build a physically sensible tree. Good clustering reduces wirelength and excessive buffer depth.

Topology

Common structures include balanced trees, useful-skew structures and clock meshes/hybrid structures depending on methodology. CTS chooses topology based on constraints, physical distribution and library cells.

Clock cells

Clock buffers/inverters are selected for drive strength, slew, area, power and balance. Excessive stages can increase latency; too few stages can create transition/cap problems.

Balance groups

Not every clock endpoint must be balanced against every other endpoint. Groups can represent logically/physically related sinks that need controlled relative arrival.

ICG handling

Integrated clock-gating cells create branches. The gated branch must be treated as part of the clock network; gating checks and enable timing also matter.

4. ROUTE CLOCK — RC, NDR, Shielding and Crosstalk

Why skew can change after routing: CTS estimates cannot perfectly predict detailed routed resistance, capacitance, via effects and coupling. Real extraction therefore changes insertion delay and relative arrival.

NDR

Non-default routing rules can use wider wires and/or larger spacing for selected clock nets. This can reduce resistance and coupling sensitivity, at the cost of routing resources.

Shielding

Shield wires or spacing can reduce capacitive coupling from aggressors into sensitive clock routes. Shielding is usually targeted at critical clock trunks rather than applied indiscriminately.

Layer choice

Higher or dedicated clock layers may provide lower RC or cleaner routing depending on technology. Layer restrictions are technology-specific.

Crosstalk

Aggressor switching can speed up or slow down a victim clock edge. Because clock arrival affects many endpoints, SI-induced delta delay can change skew and timing.

5. OPTIMIZE CLOCK — Latency, Skew, SI and Timing

Problem	Typical levers	What to avoid
High latency	Reduce unnecessary depth; improve routing RC; stronger/appropriate cells; reduce long detours; rebalance topology	Blindly upsizing every clock cell; power explosion
High positive/negative skew	Balance relevant endpoints; adjust branch topology; resize/insert cells; use balance groups/skew targets; useful skew where methodology permits	Forcing global zero skew when intentional useful skew is beneficial
Slow transition / high cap	Buffer insertion, cell resizing, fanout redistribution, routing improvement	Creating excessive buffering and power
Clock crosstalk	NDR, spacing, shielding, layer control, routing optimization, SI-aware clock optimization	Shielding every net without considering routing congestion
Setup issue related to clock arrival	Useful skew / CCD-style optimization, targeted latency changes, data-path interaction	Treating setup as a clock-only problem
Hold issue related to clock arrival	Balance/latency adjustment, useful skew, then verify hold across corners	Fixing one corner while breaking another

6. Technique-by-Technique Guide

A. Skew Groups

Use when only certain sinks/paths need controlled relative clock arrival. Useful for defining which endpoints are compared or balanced together in a methodology.

Scenario: Scenario: two functional blocks have different timing needs and should not be treated as one universal balance target.

Command/flow note: Representative categories: clock balance/group constraints and CTS balance-point definitions. Exact ICC2 syntax depends on release/flow.

B. Clock Balance Groups

Use to explicitly define groups of clock sinks/points that should be balanced relative to one another. Often useful for generated/gated clock relationships or carefully controlled domains.

Scenario: Scenario: one set of launch/capture sinks needs tight relative arrival while unrelated branches do not.

Command/flow note: Representative command family: balance-point/group constraints; verify actual command syntax in your ICC2 release.

C. ICG — Integrated Clock Gating

Use when clocks pass through integrated clock-gating cells. CTS must understand gating structure and build balanced gated branches.

Scenario: Scenario: a block has many gated registers and gated-clock branches show excessive latency/skew.

Command/flow note: Typical flow concepts: identify/propagate clock gating, preserve ICG structure, optimize gated branches and verify gating checks.

D. CCD — Concurrent Clock/Data optimization

Use when clock arrival and data-path optimization need to be considered together. It can deliberately alter clock/data paths to improve setup/hold rather than minimizing clock skew alone.

Scenario: Scenario: severe setup/hold tradeoff remains after conventional CTS optimization.

Command/flow note: Important: CCD is methodology/tool-flow dependent; enable it only when the signoff flow supports and expects it.

E. CDC — Clock Domain Crossing

CDC is primarily a functional/timing methodology problem, not a CTS skew-reduction technique. Use CDC analysis to verify synchronizers, async crossings, handshakes and clock-domain relationships.

Scenario: Scenario: data moves from clock A to clock B and the concern is metastability/functional correctness, not simply clock skew.

Command/flow note: CTS may affect physical clock quality, but CDC verification itself is separate from CTS optimization.

F. NDR — Non-Default Routing Rule

Use on selected clock nets/trunks when RC, transition, coupling or robustness requires stronger routing than default rules.

Scenario: Scenario: long clock trunk has high resistance or strong coupling; use wider wire/larger spacing where technology supports it.

Command/flow note: Representative category: define/apply routing rules to clock nets; exact syntax is tech/flow specific.

G. Shielding

Use when clock coupling is significant and sensitive clock routes need isolation from aggressors.

Scenario: Scenario: post-route SI analysis shows large clock delta delay or skew variation from nearby switching nets.

Command/flow note: Representative category: route selected clock nets with shield/spacing strategy and verify extraction/SI.

7. Representative ICC2-Style Command Map

The examples below are intentionally labeled representative. Synopsys ICC2 command options differ by release, technology file, clock-tree methodology and company flow. Always validate exact syntax with the installed ICC2 help/man pages.

Task	Representative command/category	When to use
Build/synthesize clock tree	synthesize_clock_trees / CTS build flow	Create the initial clock topology and buffering
Set CTS options	set_clock_tree_options	Control CTS behavior such as routing/optimization options used by the methodology
Define clock-tree references	set_clock_tree_references	Restrict/select reference clock cells or clock-tree resources when supported
Balance points/groups	clock balance-point/group constraint family	Control which clock points are balanced together
Clock routing	route_clock / clock-routing flow	Route synthesized clock network on physical metal
Routing rules	set_routing_rule / clock routing rule flow	Apply NDR or special routing requirements
Clock optimization	clock_opt	Optimize clock tree for timing/electrical objectives; exact stage/options depend on flow
Timing analysis	report_timing	Check setup/hold impact of clock arrival changes
Clock report	report_clock_tree / CTS report family	Inspect tree topology, latency, skew, fanout and clock cells
SI/coupling analysis	report_si / timing-SI reporting flow	Identify crosstalk-induced delay/noise where enabled

8. Scenario → Technique Decision Guide

Observed issue	First investigate	Possible technique/lever	Stage
Build skew is large	Sink clustering, topology, balance targets	Balance groups, topology/cell changes	Build
Post-route skew becomes large	RC/vias/coupling by branch	NDR, layer/spacing, shielding, routing optimization	Route
Clock latency too high	Tree depth + route RC	Cell sizing/topology + routing improvement	Build/Route/Opt
Clock transition violation	Cell drive + load	Resize/buffer/fanout redistribution	Build/Opt
Clock crosstalk high	Neighbor nets + spacing + route layers	NDR + shielding + spacing/layer control	Route/Opt
Gated branch is late	ICG location + gated branch topology	ICG-aware CTS/balance/optimization	Build/Opt
Setup/hold tradeoff persists	Clock/data interaction	Useful skew / CCD where enabled	Opt
Cross-domain issue	CDC structure and constraints	CDC verification; synchronizer/handshake methodology	STA/CDC, not CTS
Only selected endpoints need tight balance	Endpoint relationship	Skew/balance groups	Build/Opt

9. OCV / AOCV / POCV and CCS Connection to CTS

OCV

On-chip variation models account for variation in delay across the chip/cells/paths. Clock skew that looks acceptable nominally can become problematic under variation.

AOCV

Advanced OCV uses depth/distance-style variation dependence. CTS depth and physical distribution therefore matter to robust timing.

POCV

Parametric OCV models statistical/parametric variation. Clock and data paths can have different sensitivity, so signoff must evaluate the actual timing model.

CCS

Composite Current Source is a library modeling approach that captures current/voltage behavior more accurately than simple delay models, especially for waveform/SI-sensitive timing. CTS quality must ultimately be checked using the signoff timing model.

Interview connection

CTS does not 'solve OCV/POCV/CCS'. CTS builds a physically reasonable clock network; signoff STA evaluates that network under variation and waveform/SI models. Optimization may then iterate based on signoff results.

10. Practical End-to-End CTS Debug Example

1. Build-clock check

Report tree topology, sink count, fanout, transition, latency and preliminary skew.

2. If skew is high

Determine whether the problem is clustering/topology or a specific balance relationship. Use balance groups/points only where the relationship is meaningful.

3. Route-clock check

Inspect actual clock metal, RC, vias and coupling. Compare pre-route and post-route latency/skew.

4. If SI is the cause

Target the affected trunks with spacing/NDR/shielding or routing changes. Do not blindly apply special routing everywhere.

5. Optimize clock

Use cell sizing/insertion/removal, topology changes and timing-aware optimization. If the methodology supports CCD/useful skew, evaluate it for persistent setup/hold conflicts.

6. Signoff

Run MCMM timing with the intended OCV/POCV and SI/waveform models. Recheck setup, hold, clock transition, cap, skew and latency.

11. Interview Questions — Short Answers

Q: What are the three CTS stages?

Build clock creates the topology and buffering; route clock implements the clock network on metal; optimize clock tunes timing/electrical/SI behavior after or around routing.

Q: How do you reduce latency?

Reduce unnecessary tree depth and route RC, use appropriate drive strength, improve topology and routing, and remove unnecessary detours. Always check power and transition.

Q: How do you reduce skew?

First identify which endpoints should be balanced. Then improve clustering/topology, balance groups, cell sizing and branch RC. Do not assume zero skew is always the objective.

Q: How do you reduce clock crosstalk?

Increase spacing, use NDR where justified, shield sensitive trunks, choose suitable layers, and perform SI-aware routing/optimization.

Q: Is CDC a CTS technique?

No. CDC is primarily a clock-domain functional verification/constraint methodology. CTS can affect physical clock quality, but CDC itself is not a skew-reduction knob.

Q: Why use NDR?

To give selected clock routes better electrical characteristics—typically lower resistance and/or lower coupling sensitivity—when default routing is insufficient.

Q: Why shielding?

To reduce capacitive coupling from neighboring aggressors into sensitive clock routes; it costs routing resources and should be targeted.

Q: What is CCD?

A concurrent clock/data optimization approach that considers clock and data path changes together, potentially using intentional clock arrival adjustments to improve timing.

12. One-Page Interview Cheat Sheet

BUILD CLOCK: root → sinks → clustering → topology → buffers/inverters → fanout/cap/transition → balance groups → preliminary skew/latency.

ROUTE CLOCK: clock metal → layer selection → RC → NDR → spacing/shielding → coupling/SI → extracted skew/latency.

OPTIMIZE CLOCK: skew + latency + slew/cap → timing interaction → useful skew/CCD where supported → MCMM signoff.

Technique mapping: skew/balance groups = endpoint relationship; ICG = gated clock branches; CCD = clock/data co-optimization; CDC = separate domain-crossing verification; NDR = special routing; shielding = coupling reduction.

Golden interview sentence: “I first identify whether the problem originates in CTS topology, routed RC/coupling, or timing optimization. Then I choose the smallest targeted technique—balance groups for endpoint relationships, ICG-aware balancing for gated branches, NDR/shielding for clock SI, and CCD/useful skew for persistent clock-data timing tradeoffs—and finally verify MCMM signoff.”

End of guide — use this as an interview study reference, and verify exact ICC2 command syntax against the installed Synopsys ICC2 version and your project methodology.

ICC2 CTS Part-2: Commands + Tcl + Scenarios

Practical Synopsys IC Compiler II clock-tree guide for interviews and debugging

Version note

Examples below are based on documented ICC2 CTS concepts and representative ICC2 Tcl syntax. Exact switches and behavior can vary by ICC2/Fusion Compiler release and project methodology. Always confirm with your installed release using **help <command>** and your reference methodology.

1. ICC2 CTS flow at a glance

Typical clock_opt flow: build_clock → route_clock → final_opto. The build stage synthesizes/optimizes the tree; route_clock detail-routes clock nets; final_opto performs further timing-driven optimization. ICC2 also supports limiting clock_opt with -from/-to.

```
clock_opt -from build_clock -to route_clock
clock_opt
```

For a single clock tree, a standalone synthesis concept is:

```
synthesize_clock_trees -clocks [get_clocks CLK]
```

2. Before CTS: verify constraints

CTS cannot repair a bad clock definition. First verify primary clocks, generated clocks, clock groups, ICG/gating relationships and active MCMM scenarios.

```
create_clock -name CLK -period 2.0 [get_ports clk]
create_generated_clock -name CLK_DIV2 -source [get_ports clk] -divide_by 2 [get_pins UDIV/Q]
```

Scenario: a generated clock is missing. Symptoms can include incorrect timing relationships and confusing CTS behavior. Fix the constraint first; do not compensate with skew tricks.

3. Set target skew and target latency

Use **set_clock_tree_options** to specify CTS goals. ICC2 documentation supports `-target_skew` and `-target_latency`, with clock/corner-specific targeting.

```
set_clock_tree_options -target_skew 0.050 -target_latency 0.300
set_clock_tree_options -target_skew 0.050 -target_latency 0.300 -clocks [get_clocks CLK] -corners [get_corners SS]
```

Scenario A — skew too high: start by checking whether the problem is global or local. If only timing-related endpoint pairs are bad, local skew optimization or skew groups may be more useful than tightening a global target.

Scenario B — latency too high: inspect tree depth, cell delay, route RC and congestion. Do not simply reduce target latency without checking transition, power and skew.

Report the configured targets with:

```
report_clock_tree_options
```

4. Local skew optimization

Global skew compares the longest and shortest clock paths, but those endpoints may not have a timing path between them. Local skew focuses on clock relationships that matter to timing paths.

```
set_app_options -list {cts.compile.enable_local_skew true}
set_app_options -list {cts.optimize.enable_local_skew true}
```

When: global skew looks acceptable, but setup/hold paths still have poor launch/capture clock relationships.

Caution: local-skew target derivation can interact with explicit target-skew settings. Review the release documentation and reports before assuming the user target is the active target.

5. Clock skew groups

A skew group tells CTS that a selected set of endpoints has a particular balancing relationship. This is useful when only certain sinks need special balancing or latency treatment.

```
create_clock_skew_group -name sg1 -objects {reg1/CP reg2/CP reg3/CP}
report_clock_skew_groups
```

Scenario: four critical pipeline sinks need tight relative balance, while distant low-frequency sinks do not. Create the appropriate skew relationship instead of globally forcing every endpoint to the same target.

Important: skew groups are mode-specific in ICC2. If the requirement applies in multiple modes, create/check the relationship in the relevant modes.

6. Clock balance groups — do not confuse with skew groups

A **clock balance group** is used for balancing delay between clocks/clock structures. A **clock skew group** is an endpoint-focused skew relationship. The exact balance-group workflow depends on the clock-tree architecture and ICC2 feature being used.

Representative concepts include `derive_clock_balance_constraints` / `create_clock_balance_group` and related reporting. Use the exact command supported by your ICC2 release.

Interview distinction: skew group → selected sinks/endpoints. Balance group → balance relationship between clock trees/clock structures.

7. ICG — what to configure and debug

ICG cells are clock-gating elements. CTS must recognize their clock pins and correctly synthesize downstream gated branches. The enable side has library-defined clock-gating checks.

```
get_cells -hierarchical -filter "is_integrated_clock_gating_cell == true"
```

Scenario: gated branch has poor skew. First check whether the ICG itself is correctly identified as a CTS-usable cell, whether its placement is sensible, and whether downstream sinks are balanced. Then check transition/capacitance and gating checks.

Do not: fix a clock-gating constraint/connectivity problem by blindly adding clock buffers.

8. Clock cell/reference control

CTS needs an appropriate set of clock buffers/inverters and clock-related cells. Project methodology may include ICGs, muxes and other non-repeater clock elements in the reference list.

```
set_lib_cell_purpose -include cts [get_lib_cells */CLKBUF*]  
set_lib_cell_purpose -include cts [get_lib_cells */CLKINV*]
```

Use the actual library naming conventions in your design. Avoid assuming every BUF/INV is appropriate for CTS.

9. NDR — create and apply

Non-default routing rules can change width/spacing and therefore RC/coupling. They are especially useful on important clock segments, but blanket application can increase congestion.

```
create_routing_rule CLK_NDR -widths {M4 0.10 M5 0.10} -spacings {M4 0.10 M5 0.10}  
set_clock_routing_rules -net_type {clock} -rules {CLK_NDR} -min_routing_layer M4 -max_routing_layer M8
```

Important: treat the above as a syntax illustration; routing-rule syntax is particularly release/methodology sensitive. Check your installed ICC2 help before execution.

Scenario: long clock trunk has high RC and poor slew → consider NDR/higher metal. If the real problem is crosstalk → spacing/shielding may be more directly relevant.

10. NDR optimization option

ICC2 supports optimization using NDRs on critical nets during clock optimization. A documented application option is:

```
set_app_options -name clock_opt.flow.optimize_ndr -value true
```

Scenario: timing-critical clock/data nets benefit from non-default routing during optimization. Validate that NDR use does not create unacceptable congestion.

11. Shielding — practical decision

Shielding is primarily a physical/SI technique. The exact ICC2 command sequence for shield creation/assignment depends on the routing setup and release, so the safe workflow is to configure the project's shielding/routing methodology and verify it in the routed database.

Debug sequence: SI-off vs SI-on timing → identify victim clock segment → identify aggressors → check spacing/layer → apply targeted shielding or spacing → rerun extraction and SI-aware STA.

Do not shield everything. Shield the segments where SI benefit justifies routing-resource cost.

12. CCD — enable/disable and scenarios

ICC2 defines CCD as useful-skew optimization performed together with datapath optimization. The documented clock_opt control is:

```
set_app_options -name clock_opt.flow.enable_ccd -value true
# To disable:
set_app_options -name clock_opt.flow.enable_ccd -value false
```

Scenario: setup timing is difficult even after normal data optimization, and there is usable positive slack elsewhere. CCD can adjust clock arrival relationships while optimizing data.

Hold-heavy scenario: ICC2 also provides CCD hold-effort controls. Increasing hold priority can reduce setup-fixing ability, so use it only when hold is genuinely difficult.

```
set_app_options -name ccd.hold_control_effort -value high
```

13. CCD analysis/reporting concept

After CCD, inspect which clock latencies/offsets changed and whether the improvement is robust across scenarios. Useful-skew reports/commands are release-dependent; use the ICC2 help for the exact reporting command in your version.

Interview answer: 'I don't accept a CCD setup improvement until I check hold, all active scenarios, clock transition/cap, skew, and whether the latency adjustment is physically realizable.'

14. Clock route stage

To build and detail-route the clock trees without completing final optimization:

```
clock_opt -from build_clock -to route_clock
```

Scenario: you want to inspect post-route clock skew/latency before final data-path optimization. This isolates the clock build/route result from final_opto effects.

15. Full clock_opt vs staged execution

Full flow:

```
clock_opt
```

Staged:

```
clock_opt -from build_clock -to build_clock
# inspect
clock_opt -from route_clock -to route_clock
# inspect
clock_opt -from final_opto
```

The exact restart/continuation behavior depends on database state and flow setup. Save checkpoints before major CTS experiments.

16. Reporting — what to look at

Goal	Representative report	What you want to know
Clock target	report_clock_tree_options	What skew/latency targets are configured.
Skew groups	report_clock_skew_groups	Which sinks belong to which skew groups.
Timing	report_timing	Whether setup/hold paths are failing and which clock endpoints are involved.
Clock tree	clock-tree/timing reports in your ICC2 release	Topology, latency, skew, buffers, sinks and violations.
Routing/SI	tool-specific route/SI reports	Whether RC/coupling is causing clock degradation.

17. Scenario: high skew

Step 1: report skew and identify whether the bad skew is global or local.

Step 2: check whether the two endpoints actually have timing paths.

Step 3: inspect sink placement, tree depth, transition, capacitance and route RC.

Step 4: use skew groups/local skew optimization when the problem is relationship-specific.

Step 5: use NDR/shielding only if physical RC/SI analysis justifies it.

Step 6: rerun MCMM setup/hold.

Key lesson: **don't start by forcing a smaller global target.** Find the cause first.

18. Scenario: high latency

If every sink is late, inspect tree depth and cell delays. If only one region is late, inspect route length/congestion. If post-route latency jumps, inspect metal layers/vias/RC. If SI-on is much worse, inspect coupling.

Potential techniques: topology simplification, appropriate clock cell sizing, better sink clustering, route-layer changes, targeted NDR, and SI reduction.

19. Scenario: crosstalk-induced skew

Example: Branch A and Branch B were balanced before route. After SI extraction, Branch A becomes 45 ps later because it runs next to a high-switching bus.

Debug: compare SI-off/SI-on → identify aggressor → inspect spacing → reroute or increase spacing → use NDR/shielding if needed → re-extract → verify skew and timing.

Do not: add random clock buffers when the dominant problem is coupling.

20. Scenario: setup violation after CTS

First classify the violation: data delay, clock skew, clock latency, SI, or variation. If the clock relationship is unfavorable, examine whether useful skew/CCD is permitted. If the data path is dominant, optimize data rather than disturbing the entire clock tree.

After any clock adjustment, check hold in all relevant scenarios. A setup improvement is not complete until hold and DRC remain clean.

21. Scenario: hold violation after CTS

Check whether CTS created an early capture or late launch relationship. Then determine whether the violation is best fixed on the data path or clock path. Avoid globally changing skew to solve one local hold path unless the methodology calls for it.

Typical repairs include data-path delay insertion, cell sizing, route delay, or carefully controlled clock optimization/CCD depending on methodology.

22. Scenario: congestion caused by CTS

Symptoms: routing overflow, many detours, clock routes consuming critical tracks, NDR/shielding causing excessive resource usage.

Approach: reduce blanket NDR, target only critical trunks, review clock layers, improve sink clustering/placement, avoid unnecessary balancing relationships, and rerun route congestion analysis.

23. Interview-ready Tcl sequence

```
# 1. Verify clocks/constraints report_clocks # 2. Set CTS targets set_clock_tree_options -target_skew 0.050
-target_latency 0.300 # 3. Optional: local skew optimization set_app_options -list {cts.compile.enable_local_skew
true} set_app_options -list {cts.optimize.enable_local_skew true} # 4. Define a special skew relationship
create_clock_skew_group -name SG_PIPE -objects {U1/CK U2/CK U3/CK} # 5. Build + route clocks clock_opt -from
build_clock -to route_clock # 6. Inspect report_clock_tree_options report_clock_skew_groups report_timing # 7. Final
optimization clock_opt -from final_opto
```

This is an interview/learning template, not a drop-in production script. Real flows require technology-specific layers, NDRs, CTS cell lists, MCMM setup, exceptions and project app-options.

24. Command → scenario cheat sheet

Problem	Investigate	ICC2 concept/command
Need skew/latency target	CTS goal	set_clock_tree_options
Global skew doesn't correlate with timing	Path-specific skew	cts.*enable_local_skew
Only selected sinks need balance	Endpoint relationship	create_clock_skew_group
Need balance between clocks	Clock-tree relationship	clock balance group workflow
Long/high-RC clock trunk	Routing resistance	NDR / higher routing layers
Clock SI/coupling problem	Aggressors/spacing	NDR / shielding / reroute
Setup difficult after CTS	Useful skew	CCD via clock_opt
Hold difficult	Hold-aware CCD/data optimization	ccd.hold_control_effort + timing optimization
Need inspect before final optimization	Separate stages	clock_opt -from/-to

25. Final interview framework

Symptom → report → root cause → technique → trade-off → MCMM verification.

For example: 'Skew is high. I first determine whether it is global or local and whether the endpoints have a meaningful timing relationship. Then I inspect topology, cell delay, RC and SI. If it is relationship-specific, I consider a skew group/local-skew optimization; if it is RC/SI-driven, I consider routing changes/NDR/shielding. Finally I rerun setup/hold across all active scenarios.'

Sources used for command verification: Synopsys ICC2 implementation documentation and ICC2 CTS quick-reference material. The detailed command semantics should be checked against the installed ICC2 release.

ICC2 CTS Part-2 — Interview Tcl + Command → Problem → Technique

Synopsys ICC2-focused practical guide for CTS interviews and debugging

Release note

ICC2 options can vary by release and project methodology. Treat Tcl below as a practical template and verify exact syntax in your installed release with `help <command>`.

PART A — Interview-ready Tcl sequence

1. Basic CTS setup

```
set all_clocks [get_clocks *] set cts_cells [get_lib_cells {*/CLKBUF* */CLKINV*}] set_lib_cell_purpose -include cts $cts_cells set_clock_tree_options -clocks $all_clocks -target_skew 0.050 -target_latency 0.300 set_max_transition 0.10 -clock_path $all_clocks set_max_capacitance 0.300 -clock_path $all_clocks report_clock_tree_options
```

Interview: define clocks and CTS cells first, then set skew/latency targets and clock-path DRVs. Report the configuration before CTS.

2. CTS NDR + layers

```
create_routing_rule CLK_2W2S -default_reference_rule -multiplier_width 2 -multiplier_spacing 2 set_clock_routing_rules -net_type internal -rules CLK_2W2S -min_routing_layer M5 -max_routing_layer M8
```

The exact NDR name, layers and root/internal/sink split are technology-dependent. The interview point is to define routing rules together with sensible layer limits and then check congestion/SI.

3. Local skew

```
set_app_options -name cts.compile.enable_local_skew -value true set_app_options -name cts.optimize.enable_local_skew -value true
```

Use when: global skew looks acceptable but timing-relevant launch/capture relationships are poor.

4. User skew groups

```
foreach_in_collection m [get_modes] { current_mode $m create_clock_skew_group -name SG_PIPE -objects {U1/CP U2/CP U3/CP} } report_clock_skew_groups
```

Skew groups are mode-specific in ICC2 documentation. Use them when selected endpoints need a special balancing relationship.

5. Balance points

```
set_clock_balance_points -modes [all_modes] -clock [get_clocks CLK] -delay 150 -balance_points U_BLOCK/U_REG/CLK
```

Use a balance point when a particular point in the clock structure needs controlled balancing/delay. This is different from simply grouping endpoint sinks.

6. CCD

```
set_app_options -name clock_opt.flow.enable_ccd -value true set_app_options -name ccd.hold_control_effort -value high
```

CCD jointly considers clock arrival and datapath optimization for useful-skew timing closure. After CCD, always check hold, skew, latency and all MCMM scenarios.

7. NDR optimization during clock_opt

```
set_app_options -name clock_opt.flow.optimize_ndr -value true
```

Use when the flow allows timing-critical nets to benefit from non-default routing during optimization. Watch routing resources.

8. Staged clock_opt

```
clock_opt -from build_clock -to build_clock # inspect clock tree
clock_opt -from route_clock -to route_clock # inspect
post-route clock QoR
clock_opt -from final_opto
```

Interview: staged execution is useful for isolating whether a problem was introduced during build, route or final optimization.

9. Full clock_opt

```
clock_opt
```

The documented ICC2 flow uses clock_opt to synthesize/optimize the clock tree, route clocks and perform subsequent optimization stages; exact stage behavior depends on options and release.

10. Reporting sequence

```
report_clock_tree_options report_clock_skew_groups report_clock_qor -type summary
report_clock_qor -clock CLK -type latency
report_clock_qor -clock CLK -type local_skew -largest 100 report_timing -max_paths 20
report_constraints -max_transition -all_violators report_constraints -max_capacitance -all_violators
```

Match the report to the question: configuration → clock QoR → path timing → electrical DRC → SI/route analysis.

PART B — Command → Problem → Technique cheat sheet

Problem	Root-cause question	Technique	Verify
High global skew	Global or timing-relevant local?	set_clock_tree_options; topology/placement	Skew + latency + congestion
High local skew	Which launch/capture paths?	local-skew options; skew groups	Setup/hold + local skew
Selected sinks need balance	Which endpoints?	create_clock_skew_group	Mode coverage + skew
Clock-tree balance point	Where should delay be controlled?	set_clock_balance_points	Balance delay + downstream skew
High latency	Cell delay or wire RC?	tree/cell optimization; layers/NDR	Latency + transition + power
Long clock trunk	Is RC dominant?	set_clock_routing_rules + NDR	RC + congestion + slew
Crosstalk skew	SI-on vs SI-off?	spacing/NDR/shielding/reroute	SI delta + skew
Setup violation	Is clock relationship limiting?	CCD/useful skew + data opto	Setup + hold MCMM
Hold violation	Did skew cause it?	hold-aware data/clock optimization	Hold + setup MCMM
ICG issue	Gating, placement or balance?	ICG CTS handling + balance	Gating checks + skew
NDR congestion	Did routing rule cause it?	target NDR; placement/layers	Overflow + detours
Need stage isolation	Where did issue appear?	clock_opt -from/-to	Build vs route vs opto
Async clocks	Are they unrelated?	clock groups + CDC methodology	CDC correctness

PART C — Scenario-based interview answers

High skew

First classify global vs local skew and determine whether the worst endpoints have meaningful timing paths. Inspect sink placement, tree depth, transition, capacitance, RC and SI. Then choose local-skew/skew-group techniques or physical routing fixes.

High latency after route

Compare pre-route vs post-route latency. If the jump is post-route, inspect layers, vias, route length, congestion and RC. If cell delay dominates, inspect tree depth and cell choices.

SI-off passes; SI-on fails

Identify the clock victim and aggressor. Check coupling/spacing. Consider layer change, spacing, targeted NDR, shielding or aggressor reroute, then rerun SI-aware STA.

CCD improves setup but breaks hold

Inspect which clock arrivals moved. Keep the useful-skew benefit only if all required setup/hold scenarios remain clean; otherwise apply targeted clock/data repair.

NDR fixes timing but worsens congestion

The electrical benefit is consuming routing resources. Target NDR to important trunks/segments or change layers/placement instead of applying it globally.

Two clocks are asynchronous

Do not use CTS balancing to solve CDC. Establish the correct clock relationship and CDC architecture, then constrain/analyze accordingly.

PART D — 60-second interview answer

“For ICC2 CTS, I first verify clocks, generated clocks, modes/corners and clock-gating structure. Then I define clock-path DRVs, target skew and latency, select CTS reference cells and configure clock routing rules/layers. I use skew groups or balance points when a specific timing relationship needs controlled balancing. I run `clock_opt`, often staged as `build_clock`, `route_clock` and `final_opto` when debugging. For timing closure, I check clock QoR and timing, then use local-skew optimization, targeted NDR/SI fixes or CCD depending on the root cause. After every change I check setup, hold, transition, capacitance, congestion and MCMM signoff.”

PART E — Source notes

Command concepts were cross-checked against Synopsys ICC2 Implementation User Guide material and the ICC2 CTS Quick Reference Guide, including `set_clock_tree_options`, `create_clock_skew_group`, `clock_opt` staging, CCD, clock routing rules and CTS reporting. A public ICC2 Tcl script was used only as a syntax cross-check. Always validate against the installed release.